

Documentation projet

Tests Automatisés et CI/CD Agentique

Industrialisation d'un agent IA avec LangGraph, pytest, DLQ et GitHub Actions

Nom du projet	agentic-ci-langgraph
Atelier	Tests Automatisés et CI/CD Agentique
Cas d'usage	Agent LangGraph résilient avec routage conditionnel SUCCESS / DLQ
Technologies	Python 3.11, LangGraph, FastAPI, pytest, GitHub Actions
Repository GitHub	https://github.com/Otmaneelh333/project-23-06
Livrables principaux	Workflow final, documentation, tests >= 20 cas, rapport HTML, pipeline CI
Date	23 juin 2026
Auteur	Otmane EL HASSOUNI Hamza NACHAT Mohamed HACHIMI

Resume executif

Ce document presente l'industrialisation d'un agent IA sous forme de workflow code avec LangGraph. L'agent interroge une API externe, route les erreurs vers une Dead Letter Queue locale, expose une API FastAPI et valide automatiquement les comportements critiques a travers une suite de tests et une pipeline GitHub Actions.

Agentic AI - LangGraph - Resilience - DLQ - Tests automatisés - CI/CD

Table des matieres

1. Presentation generale du projet
2. Contexte, problematique et justification
3. Objectifs, perimetre et valeur projet
4. Architecture globale du systeme
5. Workflow LangGraph final
6. Gestion des erreurs et Dead Letter Queue
7. Qualite logicielle et suite de tests automatisés
8. Documentation des mocks utilisés
9. Pipeline CI/CD GitHub Actions
10. Procedure de test local
11. API FastAPI locale
12. Validation GitHub Actions et rapport HTML
13. Livrables et preuves attendues
14. KPI, criteres d'acceptation et risques
15. Recommandations d'amélioration
16. Conclusion

1. Presentation generale du projet

Le projet agentic-ci-langgraph vise a transformer un agent IA simple en composant logiciel industrialise. L'agent est implemente en Python avec LangGraph, expose en option via FastAPI et verifie automatiquement par une suite de tests pytest.

Le scenario principal consiste a recevoir une question utilisateur, appeler une API externe de type Wikipedia, produire une reponse si l'appel reussit et appliquer une strategie de repli si l'appel echoue. Cette strategie de repli enregistre l'incident dans une Dead Letter Queue locale afin de ne jamais faire planter l'application utilisateur.

Element	Description
Nom du projet	agentic-ci-langgraph
Type de projet	Workflow agentique code avec LangGraph
Objectif principal	Valider automatiquement les comportements critiques de l'agent
Architecture	StateGraph avec noeuds, etat partage et routage conditionnel
Execution locale	Python module, FastAPI et pytest
Industrialisation	Pipeline GitHub Actions avec rapport HTML en artefact

Synthese chef de projet

Le livrable ne se limite pas a un agent fonctionnel. Il prouve que l'agent est robuste, testable, auditable et maintenable, avec des preuves de validation locales et CI/CD.

2. Contexte, problematique et justification

Dans une application agentique, les dependances externes representent un risque operationnel important : indisponibilite d'API, erreurs HTTP, timeout, payload incomplet ou exception technique. Sans mecanisme de controle, une simple panne externe peut interrompre l'experience utilisateur ou generer une reponse non conforme.

Problematique

Comment garantir qu'un agent IA reste stable, conforme et traçable lorsqu'une API externe echoue, tout en automatisant les controles de qualite a chaque mise a jour du code ?

Question	Reponse projet
Qui ?	Equipe projet, responsable technique, formateur et utilisateurs de demonstration.
Quoi ?	Agent IA LangGraph avec chemin nominal, chemin fallback et DLQ.
Ou ?	Projet Python versionne dans GitHub et execute localement ou en CI.
Quand ?	A chaque execution locale, push ou pull request sur le depot GitHub.
Comment ?	Tests pytest deterministes, mocks, rapport HTML et pipeline GitHub Actions.
Pourquoi ?	Reduire le risque de regression, prouver la robustesse et securiser le livrable.

3. Objectifs, perimetre et valeur projet

3.1 Objectifs fonctionnels

- Recevoir une question utilisateur et la normaliser.
- Interroger une API externe via une facade Python securisee.
- Router automatiquement le workflow vers le succes ou la DLQ selon le statut de l'appel API.
- Produire une reponse finale uniforme, exploitable par un front-end ou une API.
- Exposer un endpoint FastAPI pour executer l'agent localement.

3.2 Objectifs non fonctionnels

- Resilience : aucune exception d'API externe ne doit casser le graphe.
- Conformite : les reponses sont verifiees avant retour utilisateur.
- Testabilite : les tests n'appellent ni Internet, ni LLM, ni Groq.
- Traçabilité : les echecs sont ecrits dans data/dlq_messages.jsonl.
- Automatisation : la CI s'execute a chaque push ou pull request.

Partie prenante	Valeur apportee
Responsable projet	Vision claire des livrables, preuves et limites.
Equipe technique	Architecture maintenable et tests reproductibles.
Equipe qualite	Rapport HTML, mocks documentes et criteres d'acceptation.
Utilisateur final	Agent qui repond ou bascule vers un fallback sans planter.

4. Architecture globale du systeme

L'architecture est basee sur un graphe d'etat LangGraph. Chaque noeud lit et enrichit un state partage. Le routage conditionnel se fait apres l'appel API afin de separer clairement le chemin nominal du chemin d'erreur.

```
User Input
-> prepare_input
-> call_api_node
-> route_after_api
  -> SUCCESS: generate_answer_node -> validate_answer_node -> final_response_node
  -> FAILURE: send_to_dlq_node -> final_response_node
```

Couche	Composant	Role
Interface	FastAPI / execution module	Recevoir une question et retourner la reponse finale.
Orchestration	LangGraph StateGraph	Piloter les noeuds et les transitions conditionnelles.
Integration externe	Wikipedia API facade	Recuperer un contexte lorsque la source est disponible.
Resilience	DLQ locale JSONL	Conserver les demandes qui ont echoue.
Qualite	pytest + mocks	Valider les comportements critiques sans dependance externe.
CI/CD	GitHub Actions	Executer les tests automatiquement et publier le rapport.

```
✓ PRJ-23-06
  > .agents
  ✓ .github\workflows
    ! ci-agent.yml
  ✓ agentic-ci-langgraph
    > .pytest_cache
    > .venv
  ✓ app
    > __pycache__
    📄 __init__.py
    📄 dlq.py
    📄 graph.py
    📄 schemas.py
    📄 server.py
    📄 tools.py
  ✓ data
    💎 .gitkeep
    {} dlq_messages.jsonl
  ✓ reports
    💎 .gitkeep
    <> report.html
  ✓ tests
    > __pycache__
    📄 test_20_business_cases.py
    📄 test_conformity.py
    📄 test_graph_failure.py
    📄 test_graph_normal.py
    💎 .gitignore
    ≡ pytest.ini
    ⓘ README.md
    ≡ requirements.txt
```

5. Workflow LangGraph final

Le workflow final est implemente dans app/graph.py. Il utilise StateGraph, START, END et add_conditional_edges. Cette structure permet d'isoler les responsabilites et d'obtenir un comportement deterministe et testable.

Noeud	Responsabilite	Sortie principale
prepare_input	Nettoie la question utilisateur et produit normalized_input.	State enrichi
call_api_node	Appelle l'API Wikipedia via call_wikipedia_api.	api_status, api_payload, api_error
route_after_api	Decide entre branche success et branche dlq.	Route conditionnelle
generate_answer_node	Extrait le resume et construit la reponse.	answer
validate_answer_node	Controle la conformite de la reponse.	is_conform, conformity_errors
send_to_dlq_node	Enregistre l'echec et prepare le fallback.	dlq_id, answer fallback
final_response_node	Construit la reponse uniforme finale.	final_response

5.1 Structure de la reponse finale

```
{
  "status": "SUCCESS | FALLBACK",
  "input": "Where Morocco is located?",
  "answer": "...",
  "is_conform": true,
  "conformity_errors": [],
  "dlq_id": null,
  "api_error": null
}
```

Point de conception important

L'API externe n'est jamais appelee directement depuis les tests metier. Elle est remplacee par des mocks, ce qui rend les resultats stables en local et dans GitHub Actions.

6. Gestion des erreurs et Dead Letter Queue

La Dead Letter Queue est la branche de securite du workflow. Lorsqu'une erreur API est detectee, l'agent ne plante pas : il ecrit une entree JSONL dans la DLQ puis renvoie une reponse de repli controlee.

Situation	Traitement	Resultat utilisateur
API disponible	Extraction du resume et validation de la reponse.	status = SUCCESS
HTTP 404	Erreur normalisee HTTP_ERROR_404 puis DLQ.	status = FALLBACK
HTTP 503	Erreur normalisee HTTP_ERROR_503 puis DLQ.	status = FALLBACK
Timeout	Erreur TIMEOUT puis DLQ.	status = FALLBACK
Exception	Erreur EXCEPTION: message puis DLQ.	status = FALLBACK

6.1 Format DLQ

```
{
  "id": "uuid",
  "timestamp": "2026-06-23T...",
  "original_prompt": "Where Morocco is located?",
  "status": "FAILED_ROUTED_TO_DLQ",
  "error": "HTTP_ERROR_503",
  "payload": {}
}
```

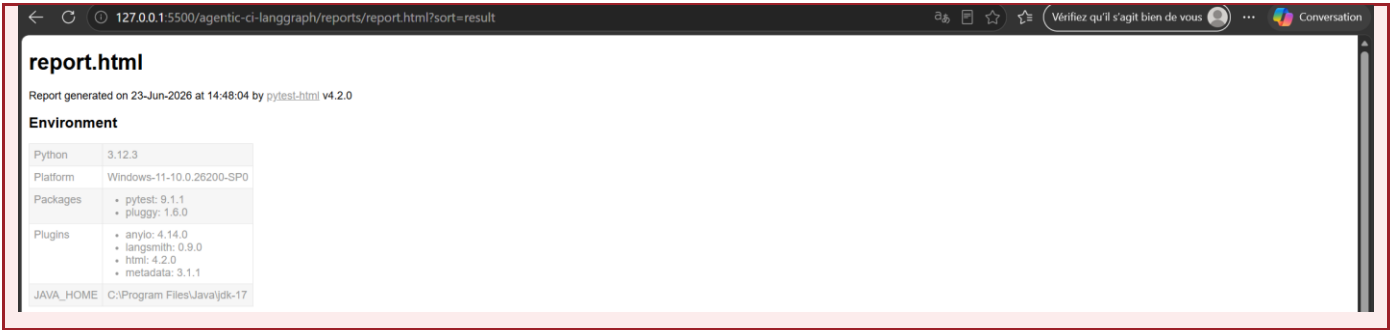
7. Qualite logicielle et suite de tests automatises

La suite de tests couvre 28 tests, dont 20 scenarios metier parametrés. Les tests valident les chemins nominaux, les erreurs, la DLQ, la conformite des reponses et les cas limites. Aucun test ne depend d'Internet, d'un LLM, de Groq ou d'une cle API.

Fichier de test	Objectif
test_graph_normal.py	Valider le chemin SUCCESS avec API mockee.
test_graph_failure.py	Valider le routage fallback et la creation DLQ.
test_conformity.py	Tester directement les regles de conformite.
test_20_business_cases.py	Executer 20 cas metier parametrés.

7.1 Couverture des scenarios

Categorie	Exemples couverts	Resultat attendu
Nominal	Question geographie, capitale, technologie, IA.	SUCCESS et reponse conforme.
Erreurs API	404, 503, timeout, exception.	FALLBACK et DLQ.
Entrees limites	Question vide, espaces, caracteres speciaux, question longue.	Traitement controle.
Payload invalide	Extract absent, vide, None, NaN.	Non-conformite ou fallback gere.
Stabilite	Cas standard repete.	Resultat reproductible.



Summary

28 tests took 00:00:01.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 28 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns ☐ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[geography_morocco]	21 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[capital_morocco]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[history]	15 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[technology]	24 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[artificial_intelligence]	22 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[empty_question]	19 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[spaces_only]	13 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_503]	11 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_404]	15 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_timeout]	21 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_exception]	24 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[payload_without_extract]	11 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[empty_extract]	10 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[answer_contains_none]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[answer_contains_nan]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[long_question]	9 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[special_characters]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[outside_domain]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[simulated_slow_api]	11 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[stable_standard_repeat]	9 ms	
Passed	tests/test_conformity.py::test_valid_answer_is_conform	1 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[empty]	2 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[too_short]	1 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[contains_none]	1 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[contains_nan]	1 ms	
Passed	tests/test_graph_failure.py::test_agent_returns_fallback_when_api_fails	65 ms	
Passed	tests/test_graph_failure.py::test_dql_entry_contains_prompt_and_failed_status	38 ms	
Passed	tests/test_graph_normal.py::test_agent_success_response_is_conform_and_not_routed_to_dql	18 ms	

← ↻ 127.0.0.1:5500/agentic-ci-langgraph/reports/report.html?sort=result

🔍 📄 ⭐

Vérifiez qu'il s'agit bien de vous

Conversation

report.html

Report generated on 23-Jun-2026 at 14:48:04 by pytest-html v4.2.0

Environment

Python	3.12.3
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• anyio: 4.14.0 • langsmith: 0.9.0 • html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-17

Summary

28 tests took 00:00:01.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 28 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns ☐ 0 Retried,
[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[geography_morocco]	21 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[capital_morocco]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[history]	15 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[technology]	24 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[artificial_intelligence]	22 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[empty_question]	19 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[spaces_only]	13 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_503]	11 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_404]	15 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_timeout]	21 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[api_exception]	24 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[payload_without_extract]	11 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[empty_extract]	10 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[answer_contains_none]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[answer_contains_nan]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[long_question]	9 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[special_characters]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[outside_domain]	12 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[simulated_slow_api]	11 ms	
Passed	tests/test_20_business_cases.py::test_twenty_business_cases[stable_standard_repeat]	9 ms	
Passed	tests/test_conformity.py::test_valid_answer_is_conform	1 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[empty]	2 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[too_short]	1 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[contains_none]	1 ms	
Passed	tests/test_conformity.py::test_invalid_answers_are_not_conform[contains_nan]	1 ms	
Passed	tests/test_graph_failure.py::test_agent_returns_fallback_when_api_fails	65 ms	
Passed	tests/test_graph_failure.py::test_dli_entry_contains_prompt_and_failed_status	38 ms	
Passed	tests/test_graph_normal.py::test_agent_success_response_is_conform_and_not_routed_to_dli	18 ms	

8. Documentation des mocks utilisés

Les mocks remplacent les dependances externes. Ils permettent de simuler exactement les comportements souhaites sans dependance reseau. Cette approche rend les tests rapides, repetables et compatibles CI/CD.

Mock	Comportement simule	Resultat attendu
API success	Retourne SUCCESS avec un extract Wikipedia valide.	Reponse non vide, conforme, sans DLQ.
API 503	Retourne FAILURE avec HTTP_ERROR_503.	Fallback et creation d'un identifiant DLQ.
API 404	Retourne FAILURE avec HTTP_ERROR_404.	Fallback controle et DLQ.
Timeout	Retourne FAILURE avec TIMEOUT.	Fallback controle et DLQ.
Exception	Retourne EXCEPTION: ...	Fallback controle et DLQ.
Payload incomplet	Payload sans extract ou extract vide.	Indisponibilite geree.
Payload non conforme	Texte contenant None ou NaN.	is_conform = False et erreur explicite.
DLQ temporaire	DLQ_PATH redirige vers tmp_path.	Aucune pollution du fichier reel pendant les tests.

Pourquoi les mocks sont essentiels ?

Les tests ne doivent pas echouer a cause d'un probleme reseau, d'un quota API ou d'un service externe indisponible. Les mocks valident la logique de l'agent, pas la disponibilite d'un fournisseur externe.

9. Pipeline CI/CD GitHub Actions

La pipeline GitHub Actions automatise les validations a chaque push ou pull request sur les branches main et master. Le fichier de workflow doit etre place a la racine du depot dans `.github/workflows/ci-agent.yml`.

```
name: Agentic LangGraph CI Pipeline

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  test-agent:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: agentic-ci-langgraph
```

Etape CI	Description
Checkout	Recupere le code source du depot.
Setup Python	Configure Python 3.11 et le cache pip.
Install dependencies	Installe les packages depuis requirements.txt.
Compile check	Compile app et tests pour detecter les erreurs syntaxiques.
Smoke test	Lance python -m app.graph.
Pytest HTML	Execute pytest et genere reports/report.html.
Upload artifact	Publie le rapport sous agentic-langgraph-test-report.

11

10. Procedure de test local

Cette procedure permet de reproduire localement la validation du projet avant le push vers GitHub.

10.1 Depuis la racine du depot

```
cd agentic-ci-langgraph
```

10.2 Creer et activer l'environnement virtuel

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

10.3 Installer les dependances

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

10.4 Compiler puis lancer l'agent

```
python -m compileall app tests
python -m app.graph
```

10.5 Lancer les tests et generer le rapport

```
pytest -v
pytest -v --html=reports/report.html --self-contained-html
```

11. API FastAPI locale

L'API locale permet de tester l'agent comme un service applicatif. Elle expose un endpoint de sante et un endpoint d'execution de l'agent.

```
uvicorn app.server:app --host 127.0.0.1 --port 8000
```

Endpoint	Methode	Usage
/health	GET	Verifier l'etat du service.
/agent/run	POST	Executer l'agent avec un JSON d'entree.

11.1 Tester l'etat de l'API

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

11.2 Tester l'agent

```
Invoke-RestMethod `
-Method Post `
-Uri http://127.0.0.1:8000/agent/run `
-ContentType "application/json" `
-Body '{"input":"Where Morocco is located?"}'
```

12. Validation GitHub Actions et rapport HTML

Après validation locale, un push vers main déclenche automatiquement la pipeline. Le responsable peut vérifier le statut du job, télécharger le rapport HTML et relancer le run si nécessaire.

12.1 Déclencher la pipeline

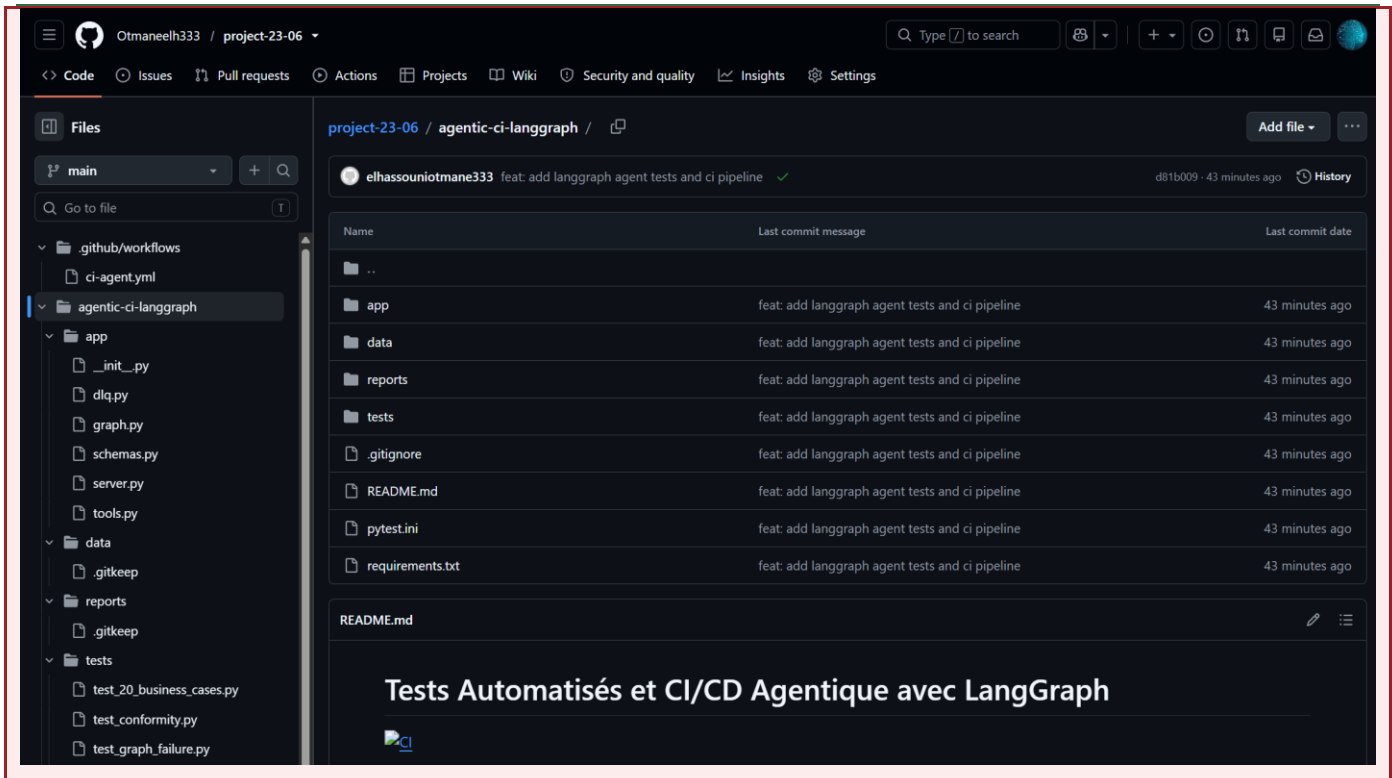
```
git add .
git commit -m "test: trigger ci pipeline"
git push origin main
```

12.2 Vérifier sur GitHub

- Ouvrir le dépôt GitHub.
- Cliquer sur l'onglet Actions.
- Ouvrir Agentic LangGraph CI Pipeline.
- Vérifier que le job test-agent est vert.
- Télécharger l'artifact agentic-langgraph-test-report pour récupérer report.html.
- Pour relancer manuellement la dernière exécution : ouvrir le run puis cliquer sur Re-run all jobs.

13. Livrables et preuves attendues

Livable attendu	Réalisation dans le projet	Preuve à insérer
Workflow final + documentation	Agent LangGraph + README + présent document.	Capture architecture projet.
Suite tests >= 20 cas	28 tests pytest dont 20 cas métier paramétrés.	Capture rapport local pytest.
Rapport de tests	reports/report.html généré localement et en CI.	Capture rapport HTML / artifact.
Pipeline CI fonctionnelle	GitHub Actions vert sur main.	Capture badge vert et job test-agent.
Documentation des mocks	Section 8 avec tableau de mocks.	Tableau inclus dans le document.



14. KPI, criteres d'acceptation et risques

KPI / Critere	Definition	Objectif
Taux de tests passes	Nombre de tests passes / nombre total.	100% avant livraison.
Couverture metier minimale	Nombre de cas metier paramétrés.	>= 20 cas.
Disponibilite fallback	Capacite a repondre sans crash si API echoue.	100% des erreurs testees.
CI verte	Dernier run GitHub Actions reussi.	Badge vert.
Rapport telechargeable	Artifact HTML disponible apres le run.	Disponible dans Actions.
Secret management	Aucune cle API requise pour les tests.	0 secret obligatoire.

14.1 Risques et parades

Risque	Impact	Parade
Workflow mal place	GitHub Actions ne se lance pas.	Mettre ci-agent.yml dans .github/workflows a la racine.
Tests dependants d'Internet	Echecs aleatoires.	Utiliser monkeypatch et mocks.
Fichiers generes poussees	Depot pollue.	Configurer .gitignore pour reports/*.html et data/*.jsonl.
Erreur d'import en CI	Tests echouent sur GitHub.	Configurer working-directory: agentique-ci-langgraph.
Cle Groq requise	CI bloquee sans secret.	Garder USE_LLM=false et aucun appel LLM dans les tests.

15. Recommandations d'amélioration

- Ajouter une visualisation graphique du graphe LangGraph dans la documentation.
- Ajouter un badge CI dans le README du depot.
- Ajouter une couverture de code avec pytest-cov si demande par l'equipe qualite.
- Ajouter un mode DLQ distant via Webhook.site ou API interne en plus du fichier JSONL local.
- Ajouter une route LLM optionnelle avec Groq uniquement hors tests, en gardant les tests deterministes.
- Ajouter une matrice de tests de regression pour comparer les sorties attendues et obtenues.

16. Conclusion

Le projet agentic-ci-langgraph fournit une implementation professionnelle d'un agent LangGraph resilient. L'architecture separe clairement l'entree utilisateur, l'appel API, le routage conditionnel, la validation de conformite, la DLQ et la reponse finale.

La suite de tests, les mocks et la pipeline GitHub Actions apportent les preuves attendues pour un livrable industriel : le workflow est fonctionnel, teste, documente, monitorable et valide automatiquement a chaque mise a jour.

Conclusion operationnelle

Le projet repond aux livrables attendus : workflow final et documentation, suite de tests superieure a 20 cas avec rapport, pipeline CI fonctionnelle avec badge vert et documentation complete des mocks utilises.

Arborescence cible

```

project-23-06/
├── .github/
│   └── workflows/
│       └── ci-agent.yml
├── agentic-ci-langgraph/
│   ├── app/
│   │   ├── graph.py
│   │   ├── tools.py
│   │   ├── dlq.py
│   │   ├── schemas.py
│   │   └── server.py
│   ├── tests/
│   │   ├── test_graph_normal.py
│   │   ├── test_graph_failure.py
│   │   ├── test_conformity.py
│   │   └── test_20_business_cases.py
│   ├── data/
│   ├── reports/
│   ├── requirements.txt
│   ├── pytest.ini
│   └── README.md

```

Checklist de demonstration

Etape	Action	Resultat attendu
1	Ouvrir le projet dans VS Code.	Structure du projet visible.
2	Activer .venv et installer requirements.txt.	Dependances installees.
3	Executer python -m compileall app tests.	Aucune erreur de compilation.
4	Executer python -m app.graph.	Reponse SUCCESS ou FALLBACK controlee.
5	Executer pytest -v.	Tous les tests passent.
6	Generer reports/report.html.	Rapport HTML disponible.
7	Pousser vers main.	Pipeline GitHub Actions declenchee.
8	Verifier test-agent.	Job vert et artifact disponible.